

Table of Contents

Studium — Evaluation and Assessment Harness Specification

Subsystem 6 of 7. Version 1.0. Status: build-ready.

Written against data layer v1.1 (with v1.2 pending, four additions from this spec joining the batch), agent runtime v1.0 with v1.0.1 patch applied (with v1.1 pending), retrieval v1.0 (with v1.1 pending), frontend v1.0, and ingestion v1.0. The nineteen items pending across those specs are noted but do not affect the evaluation boundary. This subsystem formalizes what earlier subsystems referred to as forward-referenced work: prompt regression (subsystem 2 §23), retrieval quality measurement (subsystem 3 §19), reviewer tooling for content_review_queue (subsystem 5 §12), and the summative assessment surface (subsystem 4 §11.3).

1. Overview

This subsystem carries two distinct concerns under a single name. The first is **learner-facing assessment**: the summative examination flow that grades a learner formally against a subject, distinct from the formative practice checks that run inside sessions. The second is **system-facing evaluation**: the regression harness that keeps agents honest as prompts change, models swap, and the corpus grows. Both use similar infrastructure — structured problem shapes, rubrics, grading, scored outcomes — but serve different actors and answer different questions.

The two share infrastructure because they share a computational shape: input, expected output, actual output, verdict. What differs is who the input is authored by (learner-facing: the reviewer, ahead of time; system-facing: the reviewer, ahead of time — same shape, different purpose), who the actual output comes from (learner-facing: the learner in real time; system-facing: an agent under test), and what the verdict feeds (learner-facing: mastery model and credentialing; system-facing: regression gates and metric dashboards).

A senior engineer with all five prior specs and this document should be able to build the evaluation harness end-to-end: golden datasets authored in YAML and materialized to the database, per-agent regression runs that gate prompt changes, retrieval quality measurement against hand-labelled relevance judgments, a reviewer surface that consumes both content_review_queue (system-facing) and ingestion_review_queue (already spec'd in subsystem 5), and the summative assessment flow that surfaces to the learner as a proctored examination.

The subsystem is deliberately last-before-infrastructure in the spec sequence. It depends on every prior subsystem's outputs and produces its own outputs (regression gates, quality

metrics) that inform infrastructure decisions (which prompts get deployed, which models get promoted, when a subject can be marked ready for credentialing).

2. Scope and non-goals

In scope.

- Golden datasets: structure, versioning, curation workflow, authoring format.
- Per-agent evaluation: what metrics apply to each of the seven agents, what thresholds gate deployment.
- Retrieval quality evaluation: recall@k, precision@k, reranker A/B methodology, provider swap gating.
- The prompt regression workflow: how a prompt change gets tested before deploy, how differences are surfaced, who approves.
- Content quality reviewer tooling: the surface for reviewing content_review_queue rows.
- Summative assessment flow: closed-book examination, strict grading, portfolio artifacts.
- Credentialing: what constitutes demonstrated mastery, how portfolio items get produced, how they get signed and verified.
- The reviewer role: what a reviewer can do, what surfaces they need, escalation semantics.
- Cost accounting for evaluation runs: they cost real money because they call real LLMs.
- Testing strategy: same three-tier structure, adapted for a subsystem whose product is testing infrastructure.

Explicitly out of scope.

- The Evaluator agent's runtime behavior. That lives in subsystem 2 §12. This subsystem uses the Evaluator as one of its measured entities and calls it during grading, but doesn't respect how the Evaluator itself works.
- The content_review_queue schema. That's data layer §6.13. This subsystem provides the reviewer surface that consumes it; the schema stays where it is.
- The ingestion_review_queue schema. That's subsystem 5's addition. Same relationship — this subsystem's reviewer surface reads from it, doesn't change it.
- Learner-facing formative checks (the comprehension checks during a lecture, the practice problems in the bench). Those are subsystem 2 §10 (Lecturer) and subsystem 2 §12 (Evaluator's grade_check and grade_practice). Summative assessment is the distinct thing this subsystem covers.
- Automated prompt authoring or LLM-in-the-loop dataset generation. Golden datasets are authored by humans, for the reasons named in §3. Tools that suggest edits are v1.1 candidates; automated authoring is refused entirely.
- Comparative benchmarking against other AI tutoring products. Studium's evaluation is against its own goals, not against competitors. Comparative studies

would require different methodology (blinded human evaluators, controlled subject matter, IRB approval for learner participation) and are a v2 concern if at all.

- The reviewer as a role for multiple people. MVP has one reviewer (Yannis). The role model supports multiple reviewers but the workflow spec assumes one; multi-reviewer coordination (assignment, conflict resolution, review-of-review) is a v1.1 candidate.

3. Design principles

Evaluation is measurement, not opinion. Every evaluation result is numeric where possible, categorical where necessary, and reproducible in both cases. “The Lecturer’s output feels better after the prompt change” is not an evaluation result. “The Lecturer’s segments cited from provided passages in 94% of golden dataset entries after the change vs. 87% before” is. If a quality dimension cannot be measured, it cannot be gated on; it can only inform manual reviewer judgment.

Golden datasets are authored, not generated. Every dataset entry is written by a human — the reviewer, sometimes with domain-expert input — with intent about what it tests. LLM-assisted dataset generation is refused for the same reason it was refused in ingestion’s license classification (subsystem 5 §11): confident wrong classifications compound. A dataset entry that plausibly-but-incorrectly tests a property produces regression signals against a wrong target, which is worse than no signal. When authoring capacity is limited, the response is smaller datasets, not automated ones.

Assessment is closed-book. During summative assessment, tutorial primitives are disabled, retrieval from the learner side is disabled, hint ladders are unavailable, and the Evaluator applies its strictest grading mode. The learner works from what they know, on problems they have not previously seen. This is the epistemological difference between formative practice (which is a rehearsal, done open-book with all supports) and summative assessment (which is a demonstration, done closed-book against unseen material). Any softening of the closed-book rule turns assessment into practice-with-different-branding and loses the credentialing value.

Regression is required, not optional. Any prompt change, model swap, or retrieval algorithm change runs the affected golden datasets before deploy. Regression failures block deploy until reviewed. This is what subsystem 2’s discipline about test coverage prevents in tests; it’s what this subsystem’s discipline about golden datasets prevents in production. A change that ships without regression against its scope is not an authorized change.

Credentialing requires demonstrated mastery on unseen problems. Practice problems in a session are formative; they contribute to the mastery estimate but do not credential. Credentialing requires summative assessment on problems drawn from a pool the learner has not previously encountered. This is the mechanism that lets a portfolio item claim “the learner has demonstrated mastery of X” rather than “the learner has practiced X extensively.” The distinction is what makes portfolios verifiable rather than promotional.

The reviewer is a first-class actor. The queue surfaces (content review, ingestion review, regression outcomes) all feed one interface designed for the reviewer’s workflow. Not a

generic admin panel; a specific set of tools that make the review-decide-record loop fast. The reviewer's time is the binding constraint on system quality at MVP scale; the tooling reflects that.

Prompt versions are what get evaluated, not agents. An agent's behavior is a product of its prompt, its model, and its runtime context. Evaluation runs pin the prompt (by hash), the model (by identifier), and the runtime context (by seeded fixture), so results are comparable across runs. Comparing "the Lecturer today" against "the Lecturer last week" is meaningless without pinning these dimensions.

4. Technology stack

Golden dataset storage. YAML files under `content/evaluation/` in the repo, versioned in git alongside the prompt code. Materialized to the database via `studium eval sync` on deploy for query and reporting; source of truth stays in the repo. Same pattern as subsystem 5's authoring workflows.

Regression execution. Async workers in the FastAPI process, similar to the embedding worker in retrieval. A regression run creates an `evaluation_runs` row, iterates the dataset entries, invokes the agent under test, records results.

Grading. For dataset entries whose expected output is a structured property (e.g., "must cite at least 2 passages"), grading is deterministic — a check function evaluates the actual output against the expected properties. For entries whose expected output is a qualitative judgment (e.g., "the response should demonstrate the Socratic diagnostic sequence"), grading uses the Evaluator agent in a special "meta-grading" mode with a dataset-provided rubric.

Diff surfacing. When a regression run produces different results than the prior baseline, the reviewer sees a diff: which entries scored differently, by how much, with the actual outputs side-by-side. Rendered via the reviewer surface.

Reviewer surface. CLI primary for MVP (matches the reviewer's authoring workflow); web-based admin surface as v1.1. CLI commands mirror the subsystem 5 pattern.

Signing. Portfolio items are signed with Ed25519 via `pynacl`. Keys managed per data layer §6.10 and infrastructure spec (subsystem 7).

Testing. Meta-testing — this subsystem's tests validate that the evaluation infrastructure itself works, not that specific agents pass evaluations. That's a different set of tests, run continuously, whose outputs are the metrics this subsystem produces.

5. Schema additions required

Four additive changes needed in data layer v1.2, on top of the fifteen items already pending. All are non-breaking.

Addition 1: `golden_datasets` table.

```

CREATE TYPE golden_dataset_kind AS ENUM (
  'agent_output',      -- tests an agent's response to a fixture input
  'retrieval_quality', -- tests retrieval recall/precision against labelled
                        -- relevance
  'grading_calibration', -- tests the Evaluator against known-correct grades
  'content_quality'    -- tests generated content against reviewer standards
);

CREATE TABLE golden_datasets (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  slug        TEXT NOT NULL UNIQUE,
  kind        golden_dataset_kind NOT NULL,
  agent       agent_identity,      -- populated for agent_output kind
  version     INTEGER NOT NULL DEFAULT 1,
  description TEXT NOT NULL,
  active      BOOLEAN NOT NULL DEFAULT TRUE,
  entry_count INTEGER NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_golden_datasets_kind ON golden_datasets (kind, active);
CREATE TRIGGER trg_golden_datasets_updated_at BEFORE UPDATE ON
golden_datasets
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

Addition 2: golden_dataset_entries table.

```

CREATE TABLE golden_dataset_entries (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  dataset_id  UUID NOT NULL REFERENCES golden_datasets(id) ON DELETE
CASCADE,
  entry_index INTEGER NOT NULL,
  input       JSONB NOT NULL,      -- the fixture input to the
agent/retriever
  expected    JSONB NOT NULL,      -- expected properties or
reference output
  grading_kind TEXT NOT NULL,      -- 'deterministic' or
'meta_graded'
  rubric      JSONB,               -- populated for meta_graded
entries
  notes       TEXT,                -- authoring notes
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (dataset_id, entry_index)
);

CREATE INDEX idx_dataset_entries_dataset ON golden_dataset_entries
(dataset_id);
CREATE TRIGGER trg_dataset_entries_updated_at BEFORE UPDATE ON

```

```
golden_dataset_entries
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

Addition 3: evaluation_runs table.

```
CREATE TABLE evaluation_runs (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  dataset_id        UUID NOT NULL REFERENCES golden_datasets(id) ON DELETE
RESTRICIT,
  prompt_hash       TEXT NOT NULL,           -- matches
agent_traces.system_prompt_hash
  model             TEXT NOT NULL,
  triggered_by      UUID REFERENCES users(id) ON DELETE SET NULL,
  trigger_kind      TEXT NOT NULL,           -- 'manual', 'pre_deploy',
'scheduled', 'ci'
  status            TEXT NOT NULL DEFAULT 'running', -- 'running',
'complete', 'failed'
  aggregate_score   REAL,                   -- populated when complete
  entries_run       INTEGER NOT NULL DEFAULT 0,
  entries_passed    INTEGER NOT NULL DEFAULT 0,
  entries_failed    INTEGER NOT NULL DEFAULT 0,
  cost_usd          REAL NOT NULL DEFAULT 0.0,
  started_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  completed_at      TIMESTAMPTZ,
  metadata          JSONB NOT NULL DEFAULT '{}'::jsonb
);
```

```
CREATE INDEX idx_eval_runs_dataset ON evaluation_runs (dataset_id, started_at
DESC);
CREATE INDEX idx_eval_runs_prompt ON evaluation_runs (prompt_hash);
```

Addition 4: evaluation_results table.

```
CREATE TABLE evaluation_results (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
  run_id            UUID NOT NULL REFERENCES evaluation_runs(id) ON DELETE
CASCADE,
  entry_id          UUID NOT NULL REFERENCES golden_dataset_entries(id) ON
DELETE RESTRICIT,
  actual_output     JSONB NOT NULL,           -- what the agent produced
  score             REAL NOT NULL CHECK (score >= 0.0 AND score <= 1.0),
  passed            BOOLEAN NOT NULL,
  grading_notes     TEXT,                   -- from deterministic check or
meta-grader
  cost_usd          REAL NOT NULL DEFAULT 0.0,
  latency_ms        INTEGER,
  agent_trace_id    UUID REFERENCES agent_traces(id) ON DELETE SET NULL,
  reviewer_note     TEXT,                   -- reviewer's comment on this
result
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE INDEX idx_eval_results_run ON evaluation_results (run_id);
CREATE INDEX idx_eval_results_failures ON evaluation_results (run_id) WHERE
passed = FALSE;
CREATE INDEX idx_eval_results_entry ON evaluation_results (entry_id,
created_at DESC);
```

The `agent_trace_id` foreign key links each evaluation result back to the underlying LLM call, so a reviewer can drill from a failed evaluation to the full trace (prompt, response, cost, latency).

All four additions join the v1.2 batch. Data layer v1.2 batch now contains 19 items total (four from this spec, fifteen from prior).

6. The two purposes

6.1 Learner-facing assessment

Summative examination of a learner's mastery. Distinct in kind from the formative checks that run inside sessions.

Formative vs. summative. A comprehension check in a lecture segment (subsystem 2 §10) is formative: it happens open-book, in-context, with the option to retry, and its purpose is to inform ongoing learning. A summative assessment is a formal examination: closed-book, timed (optionally), on unseen problems, with strict grading and single submission per criterion.

When it happens. Summative assessment is not part of every session. It happens on learner initiation or on a curriculum milestone (e.g., "you've reached 0.85 mastery on all load-bearing concepts in the foundations module; want to test out?"). The Curator surfaces the offer; the learner accepts or defers.

What it produces. An `assessment_attempts` row (data layer §6.8) with per-criterion `assessment_responses` rows. If the attempt passes, a `portfolio_items` row is created (§12 of this spec) signed by the system's Ed25519 key. The portfolio item is verifiable by anyone with the public key.

What it doesn't produce. Course credit, formal credentials that any institution recognizes, or transferable grades. This is Studium's own record of what the system has verified about the learner's mastery. A student can present a portfolio item to a professor, a scholarship committee, or a graduate program; whether that audience finds it credible is not something Studium controls.

6.2 System-facing evaluation

Regression testing and quality measurement for the system's own outputs. The consumers are the reviewer (who acts on failures) and the deploy pipeline (which gates on results).

When it happens. Every prompt change triggers a regression run against the affected agent's golden datasets. Every model swap triggers a regression run against all datasets

that mention the changed model. A weekly scheduled run exercises the full dataset suite to catch drift from model provider updates.

What it produces. `evaluation_runs` and `evaluation_results` rows. Aggregate metrics feed the reviewer’s dashboard. Regression failures either block deploy (in CI mode) or produce reviewer queue items (in scheduled or manual mode).

What it doesn’t produce. A single number that summarizes “quality.” Quality is multi-dimensional; the metrics per agent measure different things (Lecturer grounding, Tutor Socratic discipline, Evaluator grading calibration) that don’t collapse to one score. Attempts to produce a single number would mislead more than they inform.

7. Golden datasets

7.1 Structure

Each dataset addresses one specific evaluation question. Entries in a dataset are small, focused, and authored with clear intent. A dataset with 20 well-authored entries is more useful than one with 200 mediocre entries.

Dataset kinds (matching the enum in §5 Addition 1):

- **agent_output.** Tests an agent’s response to a fixture input. Populated for the seven agents.
- **retrieval_quality.** Tests retrieval recall/precision against labelled relevance judgments.
- **grading_calibration.** Tests the Evaluator against known-correct grades (i.e., meta-evaluation of the grader).
- **content_quality.** Tests generated content against reviewer standards (e.g., “generated lecture segments should not exceed 400 words”). Used for content that isn’t agent-produced (e.g., extractor outputs).

7.2 Authoring format

YAML files under `content/evaluation/{dataset_slug}/`. One file per dataset, or split by entry group for large datasets.

```
dataset:
  slug: lecturer_formal_stance_grounding
  kind: agent_output
  agent: lecturer
  version: 1
  description: >
    Tests that the Lecturer, when invoked with formal stance, produces
    segments
    that cite from provided passages and avoid ungrounded claims. Covers the
    common case of lambda calculus foundational concepts.

entries:
```

```

- id: entry_001
  input:
    concept:
      title: Beta-reduction
      long_description: >
        The fundamental reduction rule of the lambda calculus, replacing
        a bound variable in a body with an argument value.
    stance: formal
    retrieved_passages:
      - id: chunk_beta_001
        text: "The reduction of a beta-redex  $(\lambda x.M)N$  proceeds by ..."
        section_path: ["Chapter 3", "3.2 Beta Reduction"]
      - id: chunk_beta_002
        text: "Consider the following example:  $(\lambda x.x+1)3$  reduces to ..."
        section_path: ["Chapter 3", "3.2 Beta Reduction"]

  expected:
    properties:
      - name: cites_provided_passages
        check: at_least_n_citations
        n: 2
        from: [chunk_beta_001, chunk_beta_002]
      - name: within_word_limit
        check: word_count_between
        min: 200
        max: 400
      - name: uses_formal_stance
        check: meta_graded
        rubric: >
          The response should use precise definitional language, present
          the
          reduction rule formally (with symbols where appropriate), and
          motivate briefly before formalizing.

    grading_kind: hybrid # some deterministic, some meta_graded
    notes: >
      This entry tests the core case. Add adversarial variants (thin
      retrieval, misleading passages) as separate entries.

```

Deterministic checks. Enumerated in `studium.eval.checks`:

- `at_least_n_citations` — count [Pn] markers, verify at least N
- `word_count_between` — count words, verify within range
- `contains_none_of` / `contains_all_of` — substring absence/presence
- `structured_output_matches_schema` — validate against a Pydantic model
- `numeric_answer_within_tolerance` — for Evaluator meta-testing
- `citation_targets_valid` — every [Pn] marker resolves to a provided passage

Adding a check kind is a code change plus test; the check function lives in `studium.eval.checks` and its name is what the YAML references.

Meta-graded checks. For properties that require judgment (Socratic tone, formal stance adherence, absence of hedging), the check invokes the Evaluator agent in a special “meta-grading” mode with the rubric provided in the YAML. The Evaluator returns a score 0-1 with a written verdict; both are recorded in `evaluation_results.grading_notes`.

The meta-grading recursion. The Evaluator grades outputs of other agents in production; the Evaluator is itself an agent whose output can be evaluated. The `grading_calibration` dataset kind exists to keep the Evaluator honest — entries whose expected output is a known-correct grade against a known-correct answer, so Evaluator regressions surface if its grading drifts. This closes the loop; no other agent needs a meta-grader because they don’t grade.

7.3 Curation workflow

Authored by the reviewer, sometimes with domain expert input. The workflow:

1. Reviewer identifies a case that needs a dataset entry — a defect surfaced in production, a regression that current datasets missed, a property they want to enforce going forward.
2. Reviewer writes the entry as YAML.
3. `studium eval validate content/evaluation/` verifies the YAML parses, references exist (agent names, check kinds), and the deterministic checks compile.
4. Reviewer commits the change to the repo alongside any related prompt or code change.
5. On merge, `studium eval sync` materializes the entry to the database.
6. The entry participates in every subsequent regression run.

Retiring entries. Entries are versioned but not deleted (`golden_dataset_entries.active` isn’t a column; the dataset itself has an active flag). When an entry becomes obsolete (the property it tested no longer applies, e.g., because the spec changed), the reviewer marks the containing dataset as `active = FALSE` and creates a successor dataset with the current version. The old dataset’s results remain queryable for historical comparison.

7.4 Dataset scale

At MVP:

- Each of the seven agents has one dataset with 10-20 entries.
- Retrieval has one dataset with 20-30 entries covering the corpus subjects.
- The Evaluator has an additional `grading_calibration` dataset with 15-25 entries.

Total: ~150-250 entries at MVP. Small on purpose. Growing to ~500 entries by the end of the first year of production use as real defects surface real gaps.

Each regression run against a dataset costs the sum of the underlying LLM calls. At MVP dataset scale and Opus 5 pricing, a full regression across all agents costs ~\$5-15 per run. Scheduled runs weekly + per-change runs = ~\$50-200/month at MVP.

8. Per-agent evaluation

Each agent's dataset addresses questions specific to what that agent does. This section enumerates what a good dataset for each agent tests.

8.1 Lecturer

What matters. Segments cite from provided passages, stay within word limits, use the requested stance, avoid making claims without grounding, produce comprehension checks when requested.

Dataset shape. Concept + stance + retrieved passages → segment. Entries cover multiple concepts, multiple stances, and adversarial cases (thin retrieval, misleading passages, unusually short concept descriptions).

Key metrics.

- Grounding rate: percentage of claims backed by a citation.
- Citation validity: percentage of [Pn] markers that resolve to a provided passage.
- Stance adherence: meta-graded, per-stance.
- Length compliance: percentage within the 200-400 word range.

Blocking threshold. Grounding rate < 90% blocks deploy. Citation validity < 100% blocks deploy (an invalid citation is a hallucinated reference; not tolerable). Others surface to reviewer for judgment.

8.2 Tutor

What matters. Runs the diagnostic sequence (classify, check context, choose move, respond, close loop), maintains Socratic discipline (asks rather than tells when appropriate), maintains tone (patient, concise, not condescending), avoids fabricating facts.

Dataset shape. Concept + learner utterance + session context → Tutor response. Entries cover the five diagnostic classifications (Vocabulary, Substance, Scope, Justification, Connection), the five pedagogical moves (direct answer, return question, worked micro-example, change of representation, "let me show you where this fails"), and adversarial cases (learner asks factually wrong question, learner is upset, learner asks question outside subject scope).

Key metrics.

- Diagnostic classification accuracy (meta-graded): does the Tutor correctly identify what class of confusion the learner has?
- Pedagogical move appropriateness (meta-graded): does the Tutor choose the move that fits the classification?

- Tone adherence (meta-graded): is the response patient, concise, non-condescending?
- Fabrication rate: does the Tutor make claims without either grounding or the “this is broader context” flag?

Blocking threshold. Fabrication rate > 5% blocks deploy. Others surface to reviewer.

8.3 Evaluator

What matters. Scores match human grader on ground-truth cases, rejects confident wrong answers, awards partial credit correctly, produces useful feedback.

Dataset shape. Answer + rubric → grade. Ground-truth grades authored by domain expert (Yannis).

Key metrics.

- Scoring accuracy: exact match with ground-truth grade.
- Direction accuracy: even when the score differs, does the Evaluator get the direction right (correct scored higher than incorrect)?
- Feedback usefulness (meta-graded): does the feedback name specific gaps the learner can act on?

Blocking threshold. Scoring accuracy < 80% blocks deploy. Direction accuracy < 95% blocks deploy (getting the direction wrong is a categorical failure). Feedback usefulness surfaces to reviewer.

8.4 Curator

What matters. Respects prerequisite unlocks, prioritizes load-bearing concepts, generates sensible session agendas, produces retrieval check prompts calibrated to prior session.

Dataset shape. Mastery snapshot + subject + prior session summary → next-topic decision or session opening. Entries cover fresh learners, mid-syllabus learners, learners with specific gap patterns.

Key metrics.

- Unlock respect: percentage of next_topic choices where all prerequisites are above 0.85.
- Load-bearing priority: does the Curator pick load-bearing concepts before their dependents?
- Agenda coherence (meta-graded): is the session agenda a sensible sequence?

Blocking threshold. Unlock respect < 100% blocks deploy (unlock failures were a spec defect I introduced in subsystem 2 §9; the deterministic post-processing catches them but the Curator shouldn't produce them in the first place).

8.5 Confusion-Tracker

What matters. Flags actual gaps, avoids noise, generates hypotheses that a Tutor can act on.

Dataset shape. Turn sequence + context → tracker decision. Entries cover clear gaps, borderline gaps, and non-gaps that shouldn't flag.

Key metrics.

- Precision: percentage of flagged entries that a reviewer confirms as real gaps.
- Recall: percentage of authored-gap entries that get flagged.
- Hypothesis quality (meta-graded): does the hypothesis point at a specific misconception rather than a generic “confused about X”?

Blocking threshold. Precision < 60% or recall < 70% blocks deploy. Noise floor and miss floor are both meaningful — noise wastes reviewer time, misses defeat the tracker's purpose.

8.6 Reviewer (agent)

What matters. Generates retrieval prompts that require production not recognition, varies prompts appropriately for card stability, doesn't leak the answer.

Dataset shape. Card state + concept + prior prompt → new prompt. Entries cover various stability/difficulty combinations.

Key metrics.

- Production requirement (meta-graded): does the prompt ask the learner to produce, or to recognize/select?
- Answer leakage: does the prompt contain the answer?
- Prompt variety (meta-graded across a sequence): does the reviewer produce different kinds of prompts across cards for the same concept?

Blocking threshold. Answer leakage > 0% blocks deploy (categorical). Others surface.

8.7 Orchestrator (intent classifier)

What matters. Correctly classifies learner utterances into intents (question, answer, comment, interrupt, next, back, primitive, end_session).

Dataset shape. Utterance + state → intent. Entries cover clear cases (unambiguous questions, unambiguous answers) and adversarial cases (rhetorical questions that aren't real questions, answers phrased as questions).

Key metrics.

- Classification accuracy: exact match with ground-truth intent.

- Confidence calibration: does the model report high confidence when correct, lower when uncertain?

Blocking threshold. Classification accuracy < 90% blocks deploy.

9. Retrieval quality evaluation

Retrieval is the one subsystem whose quality is inherently subjective — “did the retrieval return the right passages” requires knowing what “right” means for a given query. Golden datasets for retrieval encode that judgment.

9.1 Dataset shape

Each entry is a query (as `retrieve_passages` would receive it) plus a list of chunk IDs the reviewer has judged relevant. Chunks not in the relevant list may still be judged non-harmful (adjacent material) or misleading (off-topic material) — a two-tier judgment.

```
entries:
- id: retrieval_001
  input:
    concept_id: concept_beta_reduction
    stance: formal
    k: 6
    query_text: "how does beta-reduction handle variable capture"
  expected:
    relevant_chunks: [chunk_beta_001, chunk_alpha_002, chunk_capture_001]
    misleading_chunks: [chunk_eta_002] # thematically nearby but wrong for
this query
  grading_kind: deterministic
```

9.2 Metrics

- **Recall@k.** Of the k retrieved chunks, how many are in the relevant list? Score = $|\text{retrieved} \cap \text{relevant}| / |\text{relevant}|$.
- **Precision@k.** Of the k retrieved chunks, how many are relevant? Score = $|\text{retrieved} \cap \text{relevant}| / k$.
- **Misleading rate.** Of the k retrieved chunks, how many are in the misleading list? Score = $|\text{retrieved} \cap \text{misleading}| / k$. Lower is better; this metric catches the “returned nearby-but-wrong content” failure mode that raw precision misses.
- **Curated preference.** For datasets that specify preferred stances, does the retrieval respect the stance weighting? Score based on rank of the highest-scoring stance-appropriate chunk.

9.3 Reranker A/B methodology

Subsystem 3 §19 open question 3 was: does Voyage rerank-2 measurably improve ordering over raw fusion? This subsystem answers it via A/B on the retrieval dataset.

Method: 1. For each entry, run retrieval with rerank enabled and record recall@6, precision@6. 2. Same entries, rerank disabled (return top-6 of raw fusion), record same

metrics. 3. Aggregate: mean improvement per metric across the dataset. 4. Reviewer decides whether the improvement justifies the reranker's per-call cost.

The measurement is a specific action tied to a specific dataset run. Not a one-time decision — repeatable when the corpus grows, when Voyage releases new rerank versions, or when a new reranker candidate appears.

9.4 Provider swap gating

If Voyage becomes unavailable or a superior provider appears, swapping requires running the retrieval dataset against the new provider and verifying quality doesn't regress. Specifically: mean recall@6 and mean precision@6 must not drop by more than 5% (arbitrary threshold, revisitable). Misleading rate must not increase.

This gate is what prevents a provider swap from being decided on price or availability alone.

10. Content quality reviewer tooling

The reviewer's surface for consuming content_review_queue rows (system-facing content review) plus their own evaluation results.

Distinct from ingestion review. Subsystem 5 §12 covers the ingestion review surface (extractor failures, license classifications, etc.). This section covers the content review surface (generated artifact quality, grading anomalies, thin grounding events). Same reviewer, different queue, different actions.

10.1 What lands in content_review_queue

Per data layer §6.13, populated by various agent-runtime decisions:

- Lecturer segments where grounding was thin (< 3 passages, from retrieval §13)
- Evaluator grades where the mastery estimate moved by more than 0.4 in one step (from subsystem 2 §12)
- Any prompt output that failed structured-output parsing after retry (from subsystem 2 §21)
- Content filter trips (from subsystem 2 §21)

10.2 Reviewer surface (CLI)

```
studium review list                # pending content review
items
studium review list --agent lecturer  # filter by originating
agent
studium review show <queue_id>      # detail view with trace
link
studium review approve <queue_id> --note "..." # item is fine, no action
needed
studium review flag <queue_id> --to-dataset lecturer_grounding --note "..."
```

```
studium review reject <queue_id> --note "... "    # item is a defect; may
trigger content revision
```

The flag --to-dataset action. Turns a real production defect into a golden dataset entry. The reviewer sees a Lecturer segment that under-grounded; they flag it; a stub dataset entry gets created capturing the fixture input that produced the defect; the reviewer edits the stub to specify what the correct output should have been; the entry participates in future regressions. This is the mechanism by which production defects strengthen the regression suite over time.

10.3 Reviewer surface (web, v1.1)

Web-based admin surface deferred. The CLI is functional for MVP scale (one reviewer, low-volume queue). When queue volume grows or when multiple reviewers coordinate, the web surface becomes worth building.

10.4 Escalation

If a content review reveals a systemic issue (e.g., every Lecturer segment on a specific concept is thin grounding, suggesting concept_sources authoring is missing), the reviewer flags the underlying issue rather than just the individual item. This creates an ingestion review queue entry (subsystem 5) rather than staying in the content review queue — the fix is upstream, not downstream.

11. Summative assessment flow

Learner-facing formal examination. Delivered through the same infrastructure as formative practice but with strict conditions.

11.1 Trigger

Two ways an assessment starts:

Learner-initiated. From the desk, learner selects “Take an assessment on [subject]” or “Take an assessment on [concept].” Confirmation dialog explains what will happen (closed-book, timed, single submission per criterion). Learner confirms.

Curator-suggested. When the Curator detects that all load-bearing concepts in a module are above 0.85 mastery, it surfaces a suggestion at the next session start: “You’ve reached solid mastery across the foundations module. Ready to test out?” Learner accepts, defers, or declines.

11.2 The assessment session mode

A new mode: `summative_assessment` (already reserved in data layer §6.0’s `session_mode` enum). Session behavior in this mode:

- **Primitives disabled.** The command palette is unavailable. Attempting to invoke returns an error message: “Primitives are not available during assessment.”

- **Retrieval disabled from learner side.** The learner cannot see citations, cannot open passage views, cannot browse the concept graph. The system’s own retrieval for the Evaluator continues (grading needs source material for reference).
- **Hints disabled.** The lab surface’s hint ladder is not rendered. Problems present without hints.
- **Interrupts disabled.** The “raise your hand” affordance is removed. No Tutor engagement during assessment.
- **Single submission per criterion.** Once submitted, no revision. The bench’s multi-attempt cycle is replaced with a single-shot mode.
- **Timed (optional).** The assessment may have a total time budget (set by the assessment definition). A timer is visible; on expiration, unsubmitted responses are collected as-is and graded.

11.3 Assessment definition

An assessment is a named collection of problems bound to a subject and specific concepts. Authored by the reviewer as a YAML file under `content/subjects/{subject_slug}/assessments/`.

```
assessment:
  slug: lambda_calculus_foundations
  subject: lambda-calculus
  title: Lambda Calculus Foundations Assessment
  concepts:
    - beta_reduction
    - alpha_equivalence
    - church_rosser
  time_limit_minutes: 60
  passing_threshold: 0.70

problems:
  - id: problem_001
    concept: beta_reduction
    prompt: >
      Reduce the following lambda term to normal form, showing each
      beta-reduction step explicitly:  $(\lambda x. \lambda y. x)(\lambda z. z)(\lambda w. w \ w)$ 
    expected_key_points:
      - Correctly identifies the outermost redex
      - Shows the substitution step correctly
      - Arrives at the normal form  $(\lambda z. z)$ 
      - Names the K-combinator behavior in the second reduction
    rubric_criterion_id: uuid-of-rubric-criterion
    max_score: 4
    time_estimate_minutes: 8
```

11.4 Grading

The Evaluator grades assessment attempts in its `grade_assessment` mode (subsystem 2 §12). Grading uses the strictest configuration — no partial-credit heuristics beyond what

the rubric explicitly permits. The learner sees only the final scores per criterion after all criteria are submitted; per-criterion feedback is available but does not include the model answer (which stays server-side per the wire discipline from the v1.0.1 patch §6).

11.5 Result

If aggregate score meets `passing_threshold`:

- `assessment_attempts.passed` set to true
- A `portfolio_items` row is created with the assessment result
- The portfolio item is signed (§12)
- The learner is notified

If below threshold:

- Result is recorded but no portfolio item is created
- The learner sees which criteria they missed, with feedback
- The Curator schedules review of the weak areas in subsequent sessions

Retake policy: the learner can retake the assessment after 7 days minimum. Retakes draw problems from the same pool; if the pool is small enough that a retake would repeat problems, the reviewer must expand the pool before retakes are permitted. This is a manual guard rather than an automated policy.

12. Credentialing and portfolios

Portfolio items are signed, verifiable records of learner accomplishment. Backed by data layer §6.10's `portfolio_items` and `signing_keys` tables.

12.1 What gets credentialed

- Passing a summative assessment produces a portfolio item.
- Completing a full subject (all load-bearing concepts at ≥ 0.85 mastery, all summative assessments for the subject passed) produces a subject-level portfolio item.
- Concept mastery alone does not produce a portfolio item — mastery estimates are formative, portfolios are summative.

12.2 Portfolio item structure

```
{
  "learner_id": "uuid",
  "subject": "lambda-calculus",
  "kind": "assessment_pass" | "subject_completion",
  "assessment_id": "uuid",           // for assessment_pass
  "score": 0.87,
  "passing_threshold": 0.70,
  "criteria_results": [...],         // per-criterion breakdown
  "issued_at": "2026-08-23T14:30:00Z",
  "issuer": "studium.app",
  "issuer_public_key_id": "key-uuid",
}
```

```
"signature": "base64(ed25519_signature)"
}
```

12.3 Signing

Each portfolio item is signed with the current issuer key via Ed25519 (`pynacl.signing.SigningKey`). The signed content is a canonical JSON serialization of the item (excluding the signature field itself). The public key is published; verifiers can validate a portfolio item's signature against the public key without contacting Studium.

Key rotation. Signing keys rotate periodically (annually, per data layer §6.10's `signing_keys` structure). Old public keys remain published so historical portfolio items stay verifiable. New portfolio items are signed with the current key.

12.4 Verification

Public verification endpoint: `GET /api/portfolio/verify/{item_id}` returns the item and its signature. Verifiers use the public key (also published) to check the signature. Verification does not require an account or authentication.

The verifier endpoint is deliberately simple — one item at a time, one signature check. Batch verification, revocation lists, and other advanced features are v2 concerns.

12.5 What portfolios are not

Not academic credit. Not institutionally recognized credentials. Not transferable grades. A portfolio item claims that Studium has verified the learner's demonstrated mastery on unseen problems; whether an external audience finds that credible is not something Studium controls. The signing infrastructure is what makes the claim itself tamper-evident; the credibility of the underlying claim depends on the credibility of Studium as an issuer, which is a longer-term property.

13. Prompt regression workflow

The workflow that gates prompt changes. Enforces the “regression is required” principle from §3.

13.1 Change lifecycle

1. Author (any team member with access) edits a prompt. The prompt lives in code — Python string constants in the relevant agent module.
2. `studium eval affected --prompt-diff` identifies which datasets are affected. Datasets reference agents by identity; changing the Lecturer prompt affects Lecturer datasets.
3. `studium eval run --datasets <slug> --prompt-hash <new_hash>` executes a regression run against the affected datasets with the new prompt.
4. Regression completes. Aggregate results are compared against the last passing run for the same dataset.

5. If aggregate score does not regress (within tolerance defined per dataset), the change proceeds to review as a normal code change.
6. If aggregate score regresses beyond tolerance, the change is blocked pending reviewer approval. The reviewer sees a diff of entries whose results changed, with actual outputs side-by-side.
7. Reviewer either approves the regression (documenting the reason — e.g., “the prompt change intentionally changes tone; the dataset entry expects old tone; entry needs update”), requests changes to the prompt, or rejects the change.

13.2 Regression tolerance

Per dataset, defined in the dataset’s YAML:

```
dataset:
  regression_tolerance:
    aggregate_score_drop_max: 0.05      # 5% aggregate drop before blocking
    per_entry_failure_max: 2           # up to 2 previously-passing entries
may fail
```

Both must hold for a change to proceed without reviewer approval. Either violation blocks.

13.3 CI integration

The regression runs are part of CI, gated by cost. Full-suite regression on every push is prohibitively expensive at ~\$5-15/run × dozens of pushes/day. Instead:

- Push to a topic branch: no regression run (too expensive at scale).
- Push to yannis or main: affected regressions run before merge is permitted.
- Weekly scheduled: full-suite run on main to catch drift.

Regression results are visible in the CI output. Reviewer approvals are recorded as CI status checks.

13.4 The “affected datasets” detection

Automated for simple cases (prompt string changed, agent identity is known, dataset with matching agent is affected). Manual for complex cases (change to a helper function that affects multiple prompts, change to the state machine that affects Orchestrator classifier). Reviewer overrides the affected set as needed via `--datasets` flag.

14. The reviewer role

Formalized. The role exists in data layer §6.1’s `user_role` enum; this subsystem defines what it means operationally.

14.1 Permissions

A user with the reviewer role can:

- Read every table in the schema.

- Update `content_review_queue` and `ingestion_review_queue` rows (resolve, dismiss, escalate).
- Approve or block prompt-change regressions.
- Author and modify golden datasets.
- Publish subjects (transition from draft to active).
- Read learner data across all learners.

The reviewer role does not:

- Delete rows from any table (data retention is per data layer §10, not per-reviewer discretion).
- Change signing keys or issue portfolio items directly (portfolio items only issue via the summative assessment flow).
- Modify learner mastery estimates directly.

14.2 Workflow surfaces

The reviewer works across three surfaces:

- **CLI** for review queue processing, dataset authoring, regression approval.
- **The frontend admin page** (subsystem 4 v1.1 candidate) for at-a-glance queue depth and dashboard.
- **Direct database access** for cases the tools don't cover — deliberately available because the reviewer role is trusted, and locking them out of raw access would prevent them from responding to unforeseen situations.

14.3 Time cost

At MVP scale (one subject, three users), the reviewer role is estimated at 2-5 hours/week of active work: reviewing queue items, running regressions on prompt changes, adding golden dataset entries as defects surface. This grows with subject count and user count roughly linearly.

The reviewer's time is the binding constraint on system quality. When queue depth exceeds what the reviewer can process, either (a) the reviewer time increases, (b) the queue's severity thresholds tighten so fewer items surface, or (c) additional reviewers are added. The system provides queue-depth dashboards so the constraint is visible before it becomes acute.

15. Cost accounting

Evaluation costs are real. They attribute to a distinct line in the cost ledger.

15.1 Cost sources

- **Regression runs.** Sum of LLM calls for the agents under test. Attributed to the triggering user (usually the reviewer) via `evaluation_runs.triggered_by`.
- **Meta-graded checks.** Evaluator calls invoked for meta-grading. Attributed to the same triggering user.

- **Summative assessment grading.** Evaluator calls in `grade_assessment` mode. Attributed to the learner taking the assessment.
- **Scheduled regressions.** Weekly full-suite run. Attributed to the system user (per data layer §6.12).

15.2 Cost ledger integration

A new column `cost_evaluation_usd` on `cost_ledger` (data layer §6.12, joining the v1.2 batch as a fifth addition from this subsystem — updating my count to five, not four).

Wait — this changes the count. Let me update: data layer v1.2 batch grows by five items from this subsystem, not four. Total: 20 items now.

Attribution rules:

- Regression runs: `cost_evaluation_usd` on the triggering user's ledger row for the day
- Summative assessment grading: `cost_agent_usd` on the learner's ledger row (matches the existing agent grading attribution)
- Scheduled system regressions: `cost_evaluation_usd` on the system user's ledger row

15.3 Budget interaction

Evaluation runs are subject to a separate per-day cap (`user_budget_caps.daily_evaluation_usd_max`), also joining the v1.2 batch — a sixth column addition. Default value: \$10/day for MVP.

Hmm, I'm going to consolidate — rather than adding two separate columns (`cost_evaluation_usd` on ledger, `daily_evaluation_usd_max` on caps), the evaluation cost can attribute to `cost_agent_usd` since it's still agent calls. The distinction “is this evaluation or is this real learner work” lives in the `evaluation_runs` table, not in the ledger's column structure.

Reverting: no new ledger columns from this subsystem. Four additions total, not six. Total v1.2 batch: 19 items.

16. Failure semantics

Failure modes and their responses.

Failure	Response
Regression run: agent call fails	Retry per subsystem 2 §21 retry policy. Persistent failure marks the entry as <code>evaluation_results</code> with <code>passed = FALSE</code> and <code>grading_notes = "agent call failed after retries"</code> . Continues with remaining entries.
Regression run: meta-grader fails	Same as above; the entry is marked failed with a specific error.

Failure	Response
Regression run: dataset entry has invalid input	<code>evaluation_runs</code> fails immediately with a validation error. No results written.
Summative assessment: learner disconnects mid-assessment	Session state preserves; on reconnect, the learner can resume. Time budget continues to count against the total.
Summative assessment: budget cap trips mid-attempt	Assessment terminates. Attempt marked incomplete. Learner sees a message explaining what happened. No portfolio item; no penalty.
Portfolio item: signing key unavailable	Attempt fails with a specific error. Learner sees “we couldn’t issue your credential right now; it will be issued automatically once the issue is resolved.” Retry runs as a background job.
Content review queue: reviewer unavailable	Items accumulate; queue-depth dashboard reflects this. No automatic action. Eventually a system alert fires (spec’d in Infrastructure subsystem 7) if queue depth exceeds a threshold.

17. Testing strategy

Meta-testing pattern. The tests validate that the evaluation infrastructure itself works.

Tier 1 — Offline (no LLM calls, no database).

- YAML parsing: valid golden dataset YAML parses cleanly; malformed YAML raises with specific errors.
- Check function registry: every check name in a YAML entry exists in `studium.eval.checks`.
- Deterministic check correctness: for each check kind, fixtures with known inputs produce expected results.
- Regression tolerance arithmetic: `should_block(current, previous, tolerance)` returns correct results across boundary cases.
- Coverage guard on the check-registry test: fails if fewer than N check kinds are exercised.
- Portfolio item signing: `sign(payload, key)` produces a valid Ed25519 signature that verifies against the corresponding public key.
- Portfolio item canonicalization: given the same payload, the canonical JSON serialization is byte-identical across invocations.

Tier 2 — Online (requires Postgres).

- Golden dataset sync: YAML → database sync produces the expected `golden_datasets` and `golden_dataset_entries` rows.
- Evaluation run round-trip: a mocked agent call → evaluation results, `evaluation_runs` aggregate calculated correctly.

- Content review queue processing: reviewer CLI commands (approve, flag, reject) update the queue correctly and produce audit trail.
- Portfolio item creation and verification: end-to-end, portfolio item is created, signed, retrievable via verify endpoint, and verifies against public key.

Tier 3 — Paid (real LLM calls).

- Full regression run against one small dataset (~5 entries) with the real Lecturer.
- Meta-grader round-trip: real Evaluator grades a real Lecturer output against a real rubric.
- Assessment end-to-end: learner (test user) takes a small assessment, real Evaluator grades, real portfolio item is issued.

CI wiring. Same pattern. Tier 1 on every push. Tier 2 on yannis and main. Tier 3 on-demand with `STUDIUM_RUN_PAID_TESTS=1`. Separate `.github/workflows/evaluation.yml`.

18. Version history

v1.0 — 23 August 2026. Initial specification. Written against data layer v1.1 (with v1.2 pending, four additions from this spec joining the batch: `golden_datasets`, `golden_dataset_entries`, `evaluation_runs`, `evaluation_results`), agent runtime v1.0 + v1.0.1 patch, retrieval v1.0, frontend v1.0, ingestion v1.0. Locks: the dual-purpose framing (learner-facing assessment + system-facing evaluation), the golden dataset authoring workflow (YAML source of truth, database materialization), the per-agent metric set with blocking thresholds, the reranker A/B methodology answering subsystem 3 §19 open question 3, the summative assessment flow with closed-book conditions, the portfolio item signing scheme, and the reviewer role's formal permissions.

Anticipated v1.1 candidates.

- Web-based reviewer surface (currently CLI-primary).
- Multi-reviewer coordination (assignment, conflict resolution).
- Automated dataset entry generation from production defects (with reviewer confirmation).
- Comparative benchmarking methodology (if warranted).
- LLM-assisted dataset authoring suggestions (with strict reviewer confirmation, per the same posture as ingestion §11).
- Portfolio item revocation.

19. Forward references and open questions

Data layer v1.2. Four additions from this subsystem: `golden_datasets`, `golden_dataset_entries`, `evaluation_runs`, `evaluation_results`. Batch now totals 19 items.

Agent runtime. The Evaluator's meta-grading mode (§7.2) is a new invocation kind on an existing agent. Belongs in subsystem 2 v1.1 alongside other pending items.

Retrieval. Reranker A/B answered via the retrieval quality dataset (§9.3), closing subsystem 3 §19 open question 3.

Frontend. The summative assessment surface (§6.5 of frontend spec was placeholder-only) becomes fully spec'd when the assessment engine ships. Belongs in a frontend v1.1 revision.

Ingestion. No direct dependencies. Ingestion produces the sources that assessment problems reference; assessment problem authoring uses ingestion's concept graph.

Infrastructure spec (subsystem 7). Signing key management (rotation, backup, secure storage). Public key publication (where the public keys live, how verifiers find them). Queue-depth alerting (when the reviewer queue exceeds thresholds).

Open questions requiring build-time answers.

1. **Blocking threshold calibration.** The per-agent thresholds in §8 are educated guesses. Real values need calibration against real regression runs on real prompts. Expect the initial thresholds to shift within the first three months of production use.
2. **Meta-grader stability.** The meta-graded checks depend on the Evaluator's judgment being stable across runs. If the same output receives materially different meta-grades on repeated evaluation, the meta-grader itself needs regression discipline. This is what the `grading_calibration` dataset is for; effectiveness measurable after first six months of use.
3. **Cost at scale.** MVP evaluation costs are budgeted at \$50-200/month. This grows with dataset size, prompt churn, and subject count. At classroom or university tier, evaluation costs become a real line item. The scheduled-run frequency and dataset scale become tuning knobs.
4. **Multi-reviewer coordination.** MVP has one reviewer (Yannis). The workflow spec assumes serial processing. When a second reviewer joins, coordination (who takes which queue items, how disagreements resolve, how review-of-review works) becomes a real concern. Deferred until it's needed.
5. **Portfolio credibility.** Whether external audiences find Studium's portfolio items credible is a question the technical infrastructure cannot answer. Related work: getting institutional recognition (via accreditation or partnership), producing outcomes evidence (via longitudinal tracking of Studium learners' subsequent performance), building issuer reputation over time. All v2+ concerns; the signing infrastructure exists so that when credibility grows, verifiability is already in place.

End of specification

This document defines the evaluation and assessment harness for Studium in full. Golden datasets, per-agent regression with blocking thresholds, retrieval quality measurement,

content review reviewer tooling, closed-book summative assessment, signed portfolio items, and the formal reviewer role. A senior engineer with all five prior specs and this document can build the harness that keeps agents honest, produces the credentialing records the product ultimately delivers, and gives the reviewer the tools to run the quality control loop.

Next in sequence: **Subsystem 7 — Infrastructure**, the final spec. Fly.io deployment topology, secrets management, observability (Langfuse, Sentry, OpenTelemetry), backups, CI/CD, signing key management, alerts. Written against all six prior specs.